

Engineering Report

Phase 6 — Keyless CI/CD via Workload Identity Federation
SportSlotReservation · Multi-tenant SaaS for community sports-facility booking
Project: sport-slot-dev · Region: asia-south1 · Repo: chandranakkalakunta/sport-slot-reservation

Executive summary

Phase 6 delivered a fully keyless, least-privilege, Terraform-managed CI/CD pipeline. On merge to **main**, GitHub Actions authenticates to Google Cloud via Workload Identity Federation (no exported service-account keys), runs quality gates, builds and deploys the backend to Cloud Run, and deploys the frontend to Firebase Hosting — with zero manual commands. Branch protection enforces the pull-request + passing-gates path, making “committed = deployed” both true and mandatory. The pipeline was validated end to end via PR #1 (gates green, auto-deploy green, both surfaces live).

Outcomes

- **Keyless authentication** — direct WIF; the repository principalSet holds IAM directly; no JSON keys anywhere in the repo or secrets.
- **Least-privilege IAM** — every permission discovered wall-by-wall and codified in Terraform (`wif_iam.tf`); reproducible for test/prod via tfvars.
- **Enforced PR flow** — branch protection on main requires a PR plus passing backend-gates and frontend-gates checks.
- **Both surfaces automated** — Cloud Run (backend) and Firebase Hosting (frontend) deploy keylessly on merge.

Architecture

Two authentication paths share one WIF provider:

Surface	Identity path	Token
Cloud Build / Artifact Registry / Cloud Run	Direct WIF → principalSet IAM	gcloud federated credential
Firebase Hosting (REST API)	WIF → impersonate sa-firebase-admin	OAuth2 access token (~1024 chars)

Deploy flow (on push to main)

- **gates** — backend (ruff, bandit, pytest ≥ 90% coverage) and frontend (lint, test, build).
- **build-push** → Cloud Build builds the image and pushes to Artifact Registry.
- **deploy-dev** → Cloud Run deploy (runtime identity sa-cloud-run).
- **mint token** → second auth step impersonates sa-firebase-admin to mint an OAuth2 access token (`token_format=access_token`).
- **deploy hosting** → builds the frontend and deploys via the Firebase Hosting **REST API** using the minted token (no firebase-tools).

Why the Hosting deploy is unusual

Firebase’s tooling assumes either interactive login or a JSON service-account key. Our keyless direct-WIF setup has neither, which drove a sequence of deliberate engineering decisions:

- **firebase-tools cannot consume a WIF external_account credential** for deploys (its internal token manager only understands user-login or the deprecated login:ci refresh token). Confirmed via `--debug`.
- **The official action-hosting-deploy requires a JSON key** (`firebaseServiceAccount`) — incompatible with keyless. Rejected.
- **A REST-usable OAuth2 access token requires SA impersonation** — direct WIF yields a federated credential that gcloud uses internally but cannot be handed to a REST API as a bearer token.
- **Resolution:** deploy via the Hosting REST API with a token minted by impersonating `sa-firebase-admin`; translate `firebase.json` config (source/destination) to the REST schema (glob/path).

Convergence trail

Each error precisely named the next fix — the signature of correctly diagnosed, least-privilege work:

Stage	Signal	Fix
Auth	401 UNAUTHENTICATED	SA-impersonated access token
Request body	400 INVALID_ARGUMENT	Translate config to REST glob/path
Permission	403 USER_PROJECT_DENIED	serviceUsageConsumer on the SA
Result	200 / success	Released to live

Key lessons

Toolchain drift (local vs clean CI)

A clean, version-pinned CI environment surfaced five latent “works on my machine” issues that local runs hid: lint scope, security-scanner false positives, a non-hermetic test depending on ambient credentials, pnpm version drift (v9 vs v11), and a coupled Node version requirement. **Pin every toolchain version explicitly** (`packageManager`, `engines.node`, `.python-version`) so local and CI are byte-identical.

Least privilege, discovered and codified

Rather than granting a broad role and moving on, each required permission was discovered by hitting a specific wall and then added to Terraform. The result is an auditable, reproducible permission set that reads as an exact inventory of what CI does — and nothing more. The pain of discovery happens once; the result is codified forever.

Identity shifts under impersonation

Moving the Hosting deploy to SA impersonation changed the effective caller from the `principalSet` to `sa-firebase-admin`. IAM required by those REST calls had to be granted to the service account, not only the `principalSet` — a subtle but important consequence to remember when mixing direct WIF and impersonation.

Operational notes & follow-ups

- Reference runbook committed: `docs/runbooks/keyless-firebase-hosting-wif.md`.
- Phase 9 hardening: tighten project-level `storage.admin` on the `principalSet` to a bucket-scoped grant on the Cloud Build staging bucket.
- Non-blocking: Node 20 → 24 action deprecation; cosmetic `AuthContext` fast-refresh lint warning.
- The pattern (mint an SA access token via `auth@v3`, call a Google REST API directly) generalizes to any Google API whose CLI/SDK resists WIF ADC.

Status: Phase 6 complete. CI/CD is the enforced deployment path; both surfaces deploy keylessly on merge to main.

SportSlotReservation — internal engineering report. Generated for project documentation.